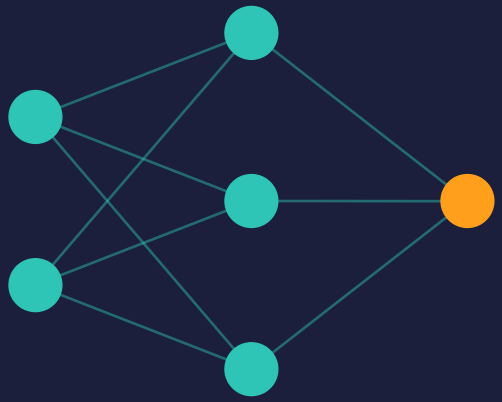
دوره یادگیری عمیق و شبکه‌های عصبی



جلسه ۸

معماری‌های مدرن CNN: از LeNet تا ResNet

VGG، مشکل تخریب در شبکه‌های عمیق و یادگیری باقیمانده (Residual Learning)

کیوان محمودی

در این جلسه

۱ رقابت ImageNet و تاریخچه

۲ LeNet و AlexNet

۳ VGG: قدرت کرنل‌های ۳×۳

۴ مشکل تخریب (Degradation)

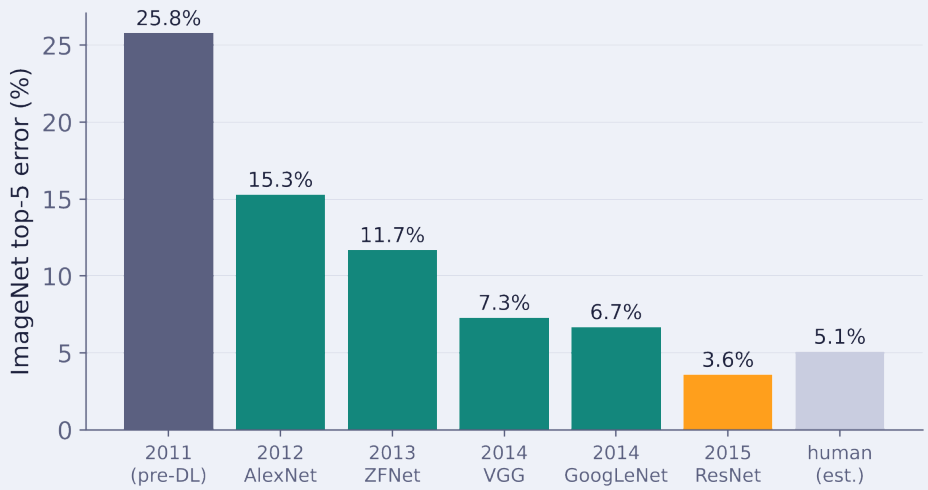
۵ ایده Residual Learning

۶ پیاده‌سازی ResNet-20

۷ Plain در برابر ResNet

۸ مدل‌های آماده torchvision

رقابت ImageNet: موتور پیشرفت CNNها



- بیش از ۱/۲ میلیون تصویر در ۱۰۰۰ کلاس
- AlexNet در ۲۰۱۲ خطا را از ۲۶٪ به ۱۵٪ رساند.
- ResNet در ۲۰۱۵ از تخمین خطای انسان (۵٪) بهتر شد.
- خطای Top-5: پاسخ درست جزو ۵ پیش‌بینی اول نباشد.

ایده کلیدی هر معماری

۱ LeNet-5 (1998)

الگوی پایه: Conv → Pool → FC؛ خواندن ارقام دست‌نویس چک‌های بانکی

۲ AlexNet (2012)

CNN عمیق روی GPU با ReLU، Dropout و Data Augmentation

۳ VGG (2014)

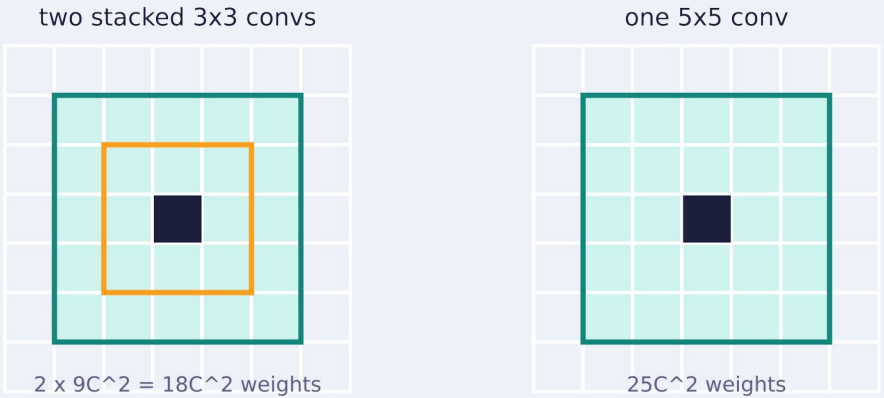
شبکه یکنواخت و عمیق فقط با کرنل‌های ۳×۳

۴ ResNet (2015)

اتصال‌های میان‌بر و آموزش شبکه‌های ۱۵۲ لایه

VGG: چرا کرنل‌های کوچک؟

- دو لایه ۳×۳ همان میدان دید یک لایه ۵×۵ را دارند.
- پارامتر کمتر: $18C^2$ در برابر $25C^2$
- یک ReLU اضافه بین دو لایه: غیرخطی بودن بیشتر
- طراحی بلوکی: در هر بلوک کانال‌ها دو برابر و ابعاد نصف



مشکل تخریب (Degradation)

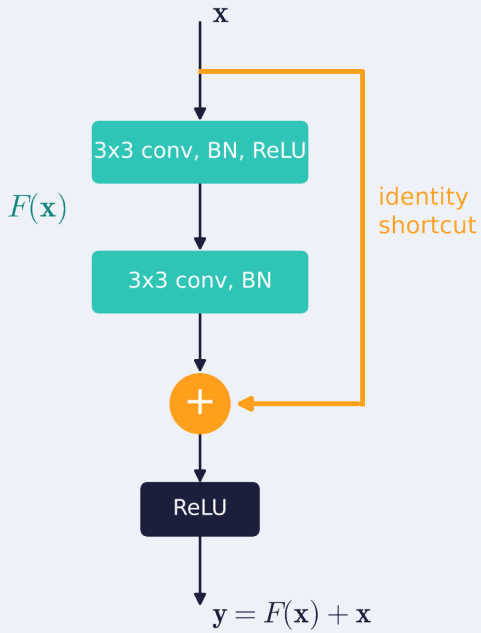
مشاهده He و همکاران

- شبکه ساده ۵۶ لایه از شبکه ۲۰ لایه بدتر بود.
- حتی خطای آموزش بیشتر بود؛ پس Overfitting نیست.
- یک مشکل بهینه‌سازی است.

استدلال

- شبکه عمیق‌تر حداقل باید به خوبی شبکه کم‌عمق باشد.
- لایه‌های اضافه کافی است تابع همانی را یاد بگیرند.
- اما یادگرفتن تابع همانی با چند لایه غیرخطی برای SGD سخت است.

Residual Learning: y = F(x) + x



- لایه‌ها فقط باقیمانده، یعنی تفاوت خروجی مطلوب و ورودی، را یاد می‌گیرند.
- اگر همانی بهینه باشد، کافی است F به صفر برود.
- مشتق خروجی نسبت به ورودی یک جمله همانی (I) دارد.
- این جمله یک بزرگراه مستقیم برای گرادیان است.
- وقتی ابعاد تغییر می‌کند: میان‌بر با Conv ۱×۱ (Projection)

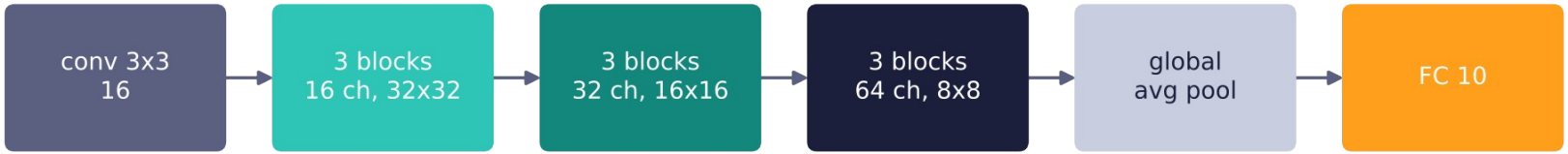
BasicBlock در PyTorch

```
class BasicBlock(nn.Module):
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.shortcut = nn.Identity()
        if stride != 1 or c_in != c_out:      # projection
            self.shortcut = nn.Sequential(
                nn.Conv2d(c_in, c_out, 1, stride, bias=False),
                nn.BatchNorm2d(c_out))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.shortcut(x))
```

- BatchNorm بایاس کانولوشن را بی‌اثر می‌کند: bias=False
- جمع با میان‌بر قبل از ReLU آخر
- با residual=False همان شبکه بدون میان‌بر (Plain) ساخته می‌شود.

ResNet-20 برای CIFAR-10



سه مرحله با ۱۶، ۳۲ و ۶۴ کانال و ۳ بلوک در هر مرحله؛ فقط ۲۷۲,۴۷۴ پارامتر

جریان گرادیان: Plain در برابر Residual

- نُرم گرادیان رسیده به لایه اول، قبل از آموزش و روی داده تصادفی:
  - ۲۰ لایه: Plain ۲/۰ و ResNet ۰/۸۸
  - ۵۶ لایه: Plain ۱۱۵۰ و ResNet ۷/۸
  - ۱۱۰ لایه: Plain ۸/۴ میلیون و ResNet ۳۷
- در شبکه ساده عمیق با BatchNorm گرادیان لایه‌های اول منفجر می‌شود؛ بهینه‌سازی ناپایدار است.
- در ResNet نُرم گرادیان با عمق فقط به‌آرامی تغییر می‌کند.
- آزمایش آموزش Plain-20 و ResNet-20 روی CIFAR-10 در نوتبوک (اجرا روی GPU)

مدل‌های آماده در torchvision.models

- resnet18 با ۱۱/۷ میلیون و resnet50 با ۲۵/۶ میلیون پارامتر
- vgg16 با ۱۳۸ میلیون پارامتر؛ بیشترش در لایه‌های FC انتهایی
- ResNet-50 به بعد: Bottleneck Block با ۱×۱ ← ۳×۳ ← ۱×۱
- MobileNet: کانولوشن Depthwise Separable برای موبایل
- EfficientNet: مقیاس‌دهی متوازن عمق، پهنا و رزولوشن
- ConvNeXt: بازطراحی CNN با ایده‌های Transformer

جمع‌بندی

- AlexNet (۲۰۱۲) آغازگر موج یادگیری عمیق بود.
- VGG: پشته کرنل‌های ۳×۳ و طراحی بلوکی یکنواخت
- شبکه‌های ساده بسیار عمیق دچار تخریب می‌شوند.
- ResNet: اتصال میان‌بر و مسیر مستقیم گرادیان
- Projection Shortcut و Bottleneck Block
- اتصال Residual امروز در تقریباً همه معماری‌ها، از جمله Transformer، حضور دارد.

تمرین

۱ پارامترهای سه لایه ۳×۳ با ۲۵۶ کانال را با یک لایه ۷×۷ مقایسه کنید.

۲ یک Bottleneck Block پیاده‌سازی و پارامترهایش را با دو BasicBlock مقایسه کنید.

۳ آزمایش Plain و ResNet را با ۵۶ لایه (n = 9) تکرار کنید.

۴ Pre-activation ResNet را بخوانید و جای ReLU را در BasicBlock تغییر دهید.